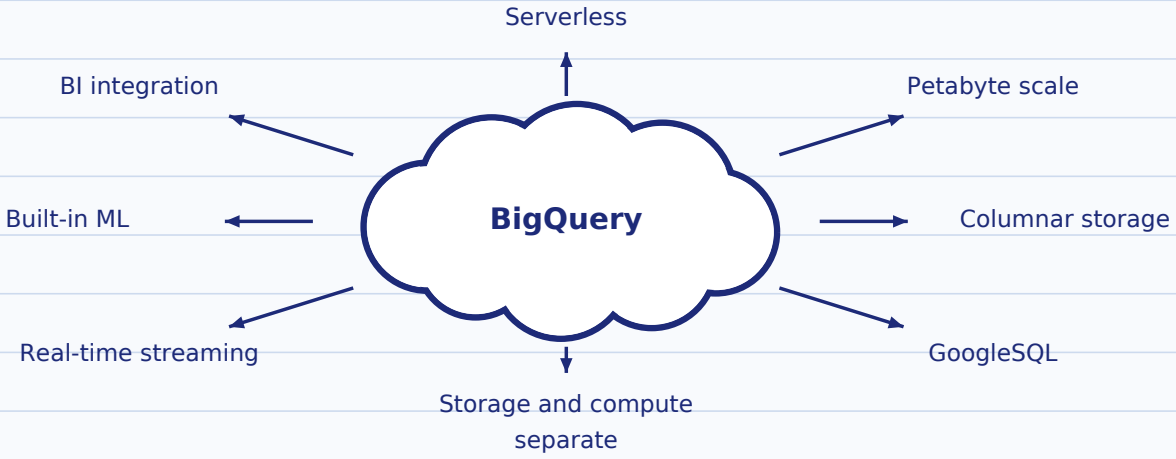
BIGQUERY NOTES

Google Cloud Platform (GCP) - Complete Detailed Notes

1. Introduction to BigQuery

Google Cloud data warehouse

Key Ideas



What is BigQuery?

- Google Cloud's **fully managed, serverless enterprise data warehouse** for analytics at scale.
- You write **SQL**; Google runs the servers, scaling, replication, patching and backups.
- Built for **OLAP** - large analytical queries over billions of rows, up to **petabyte scale**.
- **Storage and compute are separate**, so each scales independently.
- Pay for what you use: storage + queries.

Key Benefit

✓ Fast SQL analytics on huge data with **zero infrastructure** to manage.

Key Features

Core

- **Serverless** - no clusters or VMs
- **Columnar** storage, compressed
- **GoogleSQL** (standard SQL)
- Scales to **petabytes**
- Real-time **streaming** ingestion
- Storage and compute separated

Extras

- **BigQuery ML** - ML with SQL
- **Geospatial** (GIS) analysis
- **BI Engine** in-memory speed-up
- **Omni** - multi-cloud analytics
- **Analytics Hub** data sharing
- Row / column security, CMEK, time travel

OLTP vs OLAP

Aspect	OLTP (Cloud SQL, Spanner)	OLAP (BigQuery)
Purpose	Run applications and transactions	Analyze and report on data
Typical query	Many small reads / writes of single rows	Few huge scans, aggregations, joins
Storage layout	Row-oriented	**Column-oriented**
Data size	GB to TB	TB to **PB**
Latency	Milliseconds	Seconds (batch analytics)
Users	App users, services	Analysts, data scientists, BI tools

Remember: BigQuery is **not** a transactional (OLTP) database. Use it for analytics, not for app back-ends needing row-by-row updates.

Where BigQuery Fits in a Data Pipeline



Use case	Example
Business intelligence	Dashboards on sales, finance, operations
Marketing analytics	Google Analytics 4 / Ads data exported to BigQuery
Log and event analytics	Application and clickstream logs
IoT / telemetry	Sensor data streamed in real time
Machine learning	Train and predict with SQL (BigQuery ML)
Data lake / lakehouse	Query files in Cloud Storage with external / BigLake tables

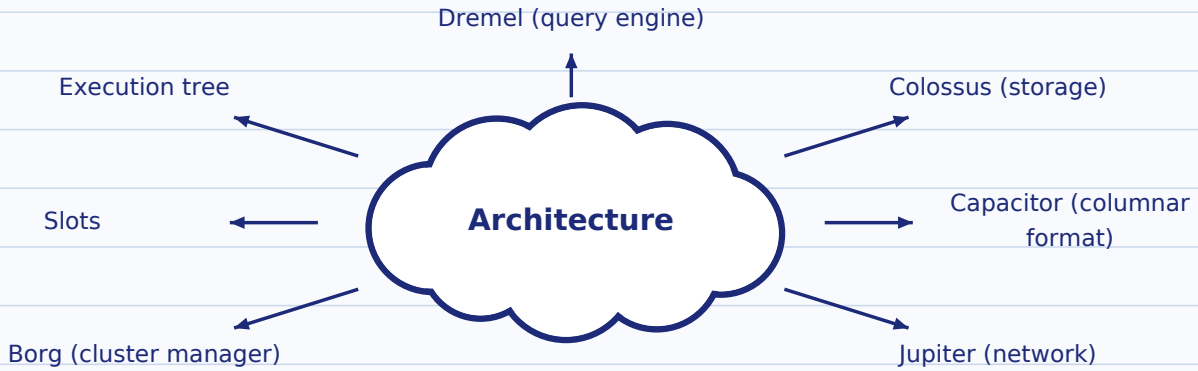
Practice Questions

- What is BigQuery? Why is it called serverless?
- Difference between OLTP and OLAP.
- Name 5 features of BigQuery.
- Where does BigQuery fit in a GCP data pipeline?

2. BigQuery Architecture

Dremel, Colossus, Jupiter, Borg

Key Ideas



Separation of Storage and Compute

Storage layer

- Data lives in **Colossus**, Google's distributed file system
- Stored in **Capacitor** columnar format, compressed
- Automatically replicated, encrypted, durable

Compute layer

- **Dremel** engine runs the SQL
- Compute power measured in **slots**
- Scheduled by **Borg** across thousands of machines

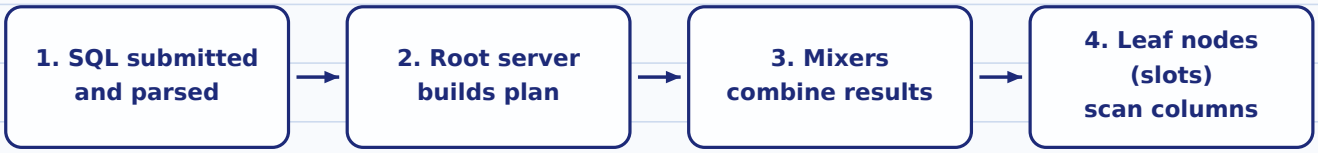
Storage
Colossus → **Jupiter network**
petabit speed → **Compute**
Dremel slots

Why it matters: You can store huge data cheaply and add compute only when queries run. Scaling one never forces scaling the other.

Internal Components

Component	Role
Dremel	Distributed query engine. Turns SQL into an **execution tree** and runs it in parallel
Colossus	Google's distributed file system - durable, replicated storage of table data
Capacitor	Columnar storage format - compresses and encodes each column separately
Jupiter	Google's high-speed datacenter network connecting storage and compute
Borg	Cluster manager that allocates compute resources (slots) to queries

How a Query Executes



- The plan is split into **stages**; each stage is run by many slots in parallel.
- Between stages data is **shuffled** through a fast in-memory shuffle tier.
- Leaf nodes read only the **columns** and **partitions** needed, then results flow up the tree to the user.

Columnar Storage

	Row-oriented	Column-oriented (BigQuery)
Layout	All values of one row stored together	All values of one column stored together
Reads for SELECT of 3 columns	Whole rows (all columns)	**Only the 3 columns**
Compression	Mixed data types, weaker	Same-type data, **very high**
Best for	Transactions	**Analytics and aggregations**

Key Point: A table has 50 columns but your query uses 3: BigQuery scans about 3/50 of the data. That is why **SELECT * is expensive** and naming columns saves money.

Slots

- A **slot** is BigQuery's unit of computational capacity (virtual CPU, memory, I/O) used to run queries.
- **On-demand**: your project draws from a shared pool (by default up to about **2,000 slots** per project).
- **Capacity (Editions)**: you buy or autoscale a number of slots for predictable performance.
- Complex queries need more slots over more stages; slot usage is visible in the **execution graph** and in INFORMATION_SCHEMA.

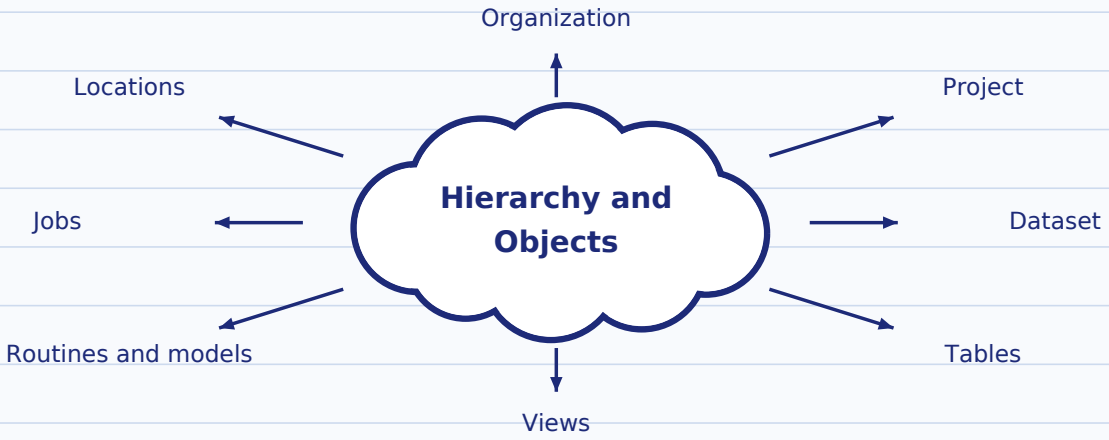
Practice Questions

- Explain separation of storage and compute.
- What are Dremel, Colossus, Capacitor, Jupiter and Borg?
- Why is BigQuery fast? (columnar + parallel)
- What is a slot?
- Why does SELECT * increase cost?

3. Resource Hierarchy and Objects

Project, dataset, tables, jobs, locations

Key Ideas



Resource Hierarchy

Organization → **Folder** → **Project** → **Dataset** → **Table / View / Routine / Model**

FULLY QUALIFIED NAME

```
`my-project.sales_ds.orders`     -- project.dataset.table

SELECT * FROM `my-project.sales_ds.orders` LIMIT 10;
```

Level	Meaning
Project	Billing, quotas and IAM boundary. All BigQuery resources belong to a project
Dataset	Container for tables, views, routines, models. Has **location**, access controls, default expiration
Table	Rows and columns of data with a schema
Routine	UDF, stored procedure or table function
Model	BigQuery ML model (created with CREATE MODEL)

Golden Rule: Project = who pays and who has permission. Dataset = where data is organized. Table = the data itself.

Table Types

Type	What it is
Native (standard) table	Data stored inside BigQuery storage
External table	Data stays in Cloud Storage, Drive, Bigtable etc.; BigQuery reads it on query
View	Saved SQL query; no data stored
Materialized view	Pre-computed, auto-refreshed query result
Table snapshot	Read-only copy of a table at a point in time
Table clone	Writable lightweight copy; pays only for changed data
Temporary / anonymous table	Holds query results; kept about **24 hours**

Jobs

Every action that works on data runs as an **asynchronous job** with a job id.

Load
files into a table

Query
run SQL

Copy
table to table

Extract
table to Cloud Storage

Note: Load, copy and extract jobs are **free** on the shared slot pool. Query jobs are billed (on-demand or slots).

Locations

Location	Meaning
Region	A specific place, e.g. asia-south1 (Mumbai), us-central1
Multi-region	A large area made of regions: **US** or **EU**

- Choose the dataset location when you **create** it - it **cannot be changed** later (copy data to move it).
- A query can only join datasets that are in the **same location**.
- Pick a location close to your data sources and users, and mind data residency rules.

INFORMATION_SCHEMA (Metadata Views)

QUERY THE METADATA

```sql
SELECT table_name, table_type, creation_time
FROM my_ds.INFORMATION_SCHEMA.TABLES;

SELECT column_name, data_type
FROM my_ds.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = "orders";

-- who ran expensive queries?
SELECT user_email, total_bytes_billed
FROM `region-us`.INFORMATION_SCHEMA.JOBS
ORDER BY total_bytes_billed DESC LIMIT 10;
```

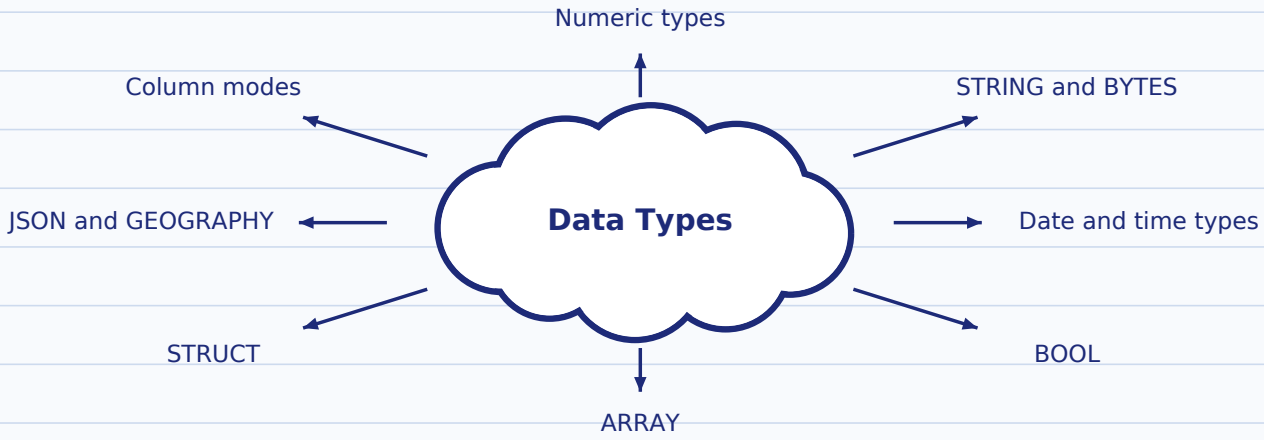
Practice Questions

- Explain the BigQuery resource hierarchy.
- Difference between region and multi-region.
- Name the table types in BigQuery.
- What are the four job types?
- Can you change a dataset location later?

4. Data Types and Schema

Scalar, nested and repeated data

Key Ideas



Data Types

Type	Description	Example
INT64	64-bit whole number (INT, BIGINT... are aliases)	42
FLOAT64	Approximate decimal number	3.14
NUMERIC / BIGNUMERIC	Exact decimal, ideal for **money**	NUMERIC "199.99"
BOOL	TRUE or FALSE	TRUE
STRING	Unicode text	"Pune"
BYTES	Raw binary data	b"abc"
DATE	Calendar date	DATE "2026-10-10"
TIME	Time of day	TIME "14:30:00"
DATETIME	Date + time, **no time zone**	DATETIME "2026-10-10 14:30:00"
TIMESTAMP	Exact moment in time (UTC)	TIMESTAMP "2026-10-10 09:00:00+00"
GEOGRAPHY	Points, lines, polygons on Earth	ST_GEOGPOINT(73.85, 18.52)
JSON	Semi-structured JSON value	JSON '{"a": 1}'
INTERVAL	Span of time	INTERVAL 3 DAY
ARRAY	Ordered list of same-type values	[1, 2, 3]
STRUCT	Group of named fields (record)	STRUCT("Asha" AS name, 25 AS age)

Column Modes

Mode	Meaning
NULLABLE	Column may contain NULL (default)
REQUIRED	Value must always be present
REPEATED	Column holds an **ARRAY** of values

Nested and Repeated Data

BigQuery works best with **denormalized** data. Instead of a separate order_items table, store items inside each order as an **ARRAY of STRUCT**. This avoids expensive joins.

SCHEMA WITH NESTED DATA

```
CREATE TABLE my_ds.orders (
  order_id INT64,
  customer STRING,
  items ARRAY<STRUCT<
    name STRING,
    qty INT64,
    price NUMERIC>>
);
```

QUERY IT WITH UNNEST

```
SELECT o.order_id, i.name, i.qty
FROM my_ds.orders AS o,
     UNNEST(o.items) AS i
WHERE i.qty > 1;

-- UNNEST flattens the array
-- one row per item
```

Remember: **STRUCT** = a record with named fields. **ARRAY** = a list. ARRAY of STRUCT = a repeated record - the BigQuery way to store one-to-many data.

Schema options: Define it manually, use **auto-detect** on load (CSV / JSON), or take it from **self-describing** formats like Avro and Parquet.

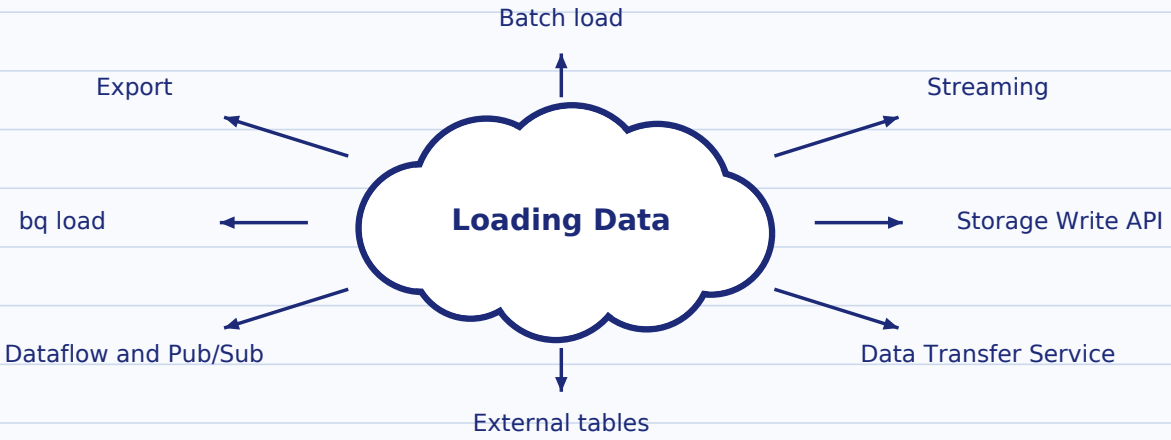
Practice Questions

- When do you use NUMERIC instead of FLOAT64?
- DATETIME vs TIMESTAMP.
- What are NULLABLE, REQUIRED, REPEATED?
- Why is nested and repeated data recommended?
- What does UNNEST do?

5. Loading and Exporting Data

Batch, streaming, transfer, external

Key Ideas



Ways to Get Data In

Method	How it works	Best for
Batch load	Load files from Cloud Storage / local into a table. **Free**	Daily / hourly files
Streaming	Insert rows continuously, available to query within seconds	Real-time dashboards, events
Storage Write API	Modern high-throughput streaming and batch API, exactly-once option	Recommended for streaming
Data Transfer Service	Scheduled, managed imports from SaaS and clouds	Google Ads, GA4, YouTube, S3 and more
Dataflow / Dataproc	Pipelines that read, transform and write to BigQuery	ETL, big transforms
Pub/Sub subscription	BigQuery subscription writes messages straight to a table	Event ingestion
External / BigLake tables	Query data in place, no load	Data lake files
Datastream	Change data capture from databases	Replicating MySQL / Postgres / Oracle

File Formats

Format	Notes
Avro	Self-describing (schema inside), fast, **preferred** for loading
Parquet / ORC	Columnar, compressed, self-describing; great for lake data
CSV	Simple; needs header handling (`skip_leading_rows`), delimiter, auto-detect or schema
JSON	Must be **newline-delimited** (one JSON object per line)
Firestore / Datastore exports	Supported for loading backups

Load Examples

bq load (CLI)

```
bq load \
  --source_format=CSV \
  --skip_leading_rows=1 \
  --autodetect \
  my_ds.orders \
  gs://my-bucket/orders.csv
```

LOAD DATA (SQL)

```
LOAD DATA INTO my_ds.orders
FROM FILES (
  format = "CSV",
  uris = ["gs://my-bucket/orders*.csv"],
  skip_leading_rows = 1
);
```

Key Point: Batch loading is **free** (you pay only for storage). Streaming and the Storage Write API have ingestion charges (with a monthly free allowance).

Streaming Ingestion

	Legacy tabledata.insertAll	Storage Write API
Throughput	Lower	**Higher**
Delivery	At-least-once (best-effort dedup with insertId)	**Exactly-once** (committed streams)
Cost	Higher	Lower, with a free monthly allowance
Status	Older	**Recommended**

Note: Streamed rows land first in a streaming buffer and are queryable almost immediately; DML on the newest buffered rows may be restricted for a short time.

External Tables and Federated Queries

- **External table**: data stays in Cloud Storage (CSV, JSON, Avro, Parquet, ORC), Google Drive or Bigtable. Slower than native and no cost saving from clustering.
- **BigLake table**: external table with fine-grained security and better performance.
- **Federated query**: run SQL on Cloud SQL, Spanner, or AlloyDB from BigQuery with EXTERNAL_QUERY.

FEDERATED QUERY

```
SELECT * FROM EXTERNAL_QUERY(
  "my-project.us.my_cloudsql_connection",
  "SELECT id, name FROM customers WHERE active = 1");
```

Tip: Native BigQuery tables are fastest. Load data in when you query it often; use external tables for occasional queries or staging.

Exporting Data

EXPORT DATA (SQL)

```
EXPORT DATA OPTIONS(
  uri = "gs://my-bucket/out/*.parquet",
  format = "PARQUET",
  overwrite = true) AS
SELECT * FROM my_ds.orders;
```

bq extract (CLI)

```
bq extract \
  --destination_format=CSV \
  my_ds.orders \
  gs://my-bucket/orders-*.csv
```

- Use a **wildcard** (*) in the URI: exports over 1 GB must be split into many files.
- Export jobs are free; you pay for Cloud Storage and any query used with EXPORT DATA.

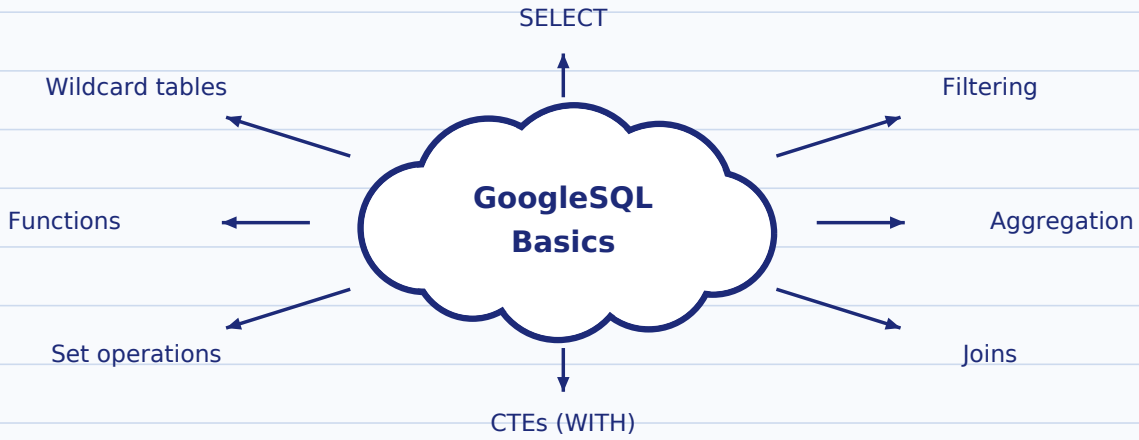
Practice Questions

- List ways to load data into BigQuery.
- Why is Avro preferred over CSV?
- Difference between batch load and streaming.
- What is the Storage Write API?
- What is an external table? A federated query?

6. GoogleSQL Basics

SELECT, joins, functions

Key Ideas



Query Structure

LOGICAL ORDER: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT

```
SELECT   region, SUM(amount) AS total      -- columns / expressions
FROM     `my-project.my_ds.sales`            -- table (backticks)
WHERE    order_date >= "2026-01-01"          -- filter rows
GROUP BY region                              -- group
HAVING   SUM(amount) > 10000                 -- filter groups
ORDER BY total DESC                          -- sort
LIMIT    10;                                 -- cap rows shown
```

Note: **GoogleSQL** (formerly Standard SQL) is the default. Legacy SQL is the old dialect - do not use it for new work. Names with dashes need **backticks**.

Handy SELECT Features

EXCEPT / REPLACE

```
SELECT * EXCEPT (password, ssn)
FROM my_ds.users;

SELECT * REPLACE (UPPER(name) AS name)
FROM my_ds.users;

SELECT DISTINCT region FROM my_ds.sales;

SELECT COUNT(*) FROM my_ds.sales;
```

WITH (CTE)

```
-- CTE: name a sub-query
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS m,
         SUM(amount) AS total
  FROM my_ds.sales
  GROUP BY m
)
SELECT * FROM monthly ORDER BY m;
```

Joins

Join	Returns
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows of the left + matches from the right (NULL if none)
RIGHT JOIN	All rows of the right + matches from the left
FULL OUTER JOIN	All rows from both sides
CROSS JOIN	Every combination (cartesian product) - use with care

EXAMPLE

```
SELECT c.name, SUM(o.amount) AS spend
FROM my_ds.orders AS o
LEFT JOIN my_ds.customers AS c
  ON o.customer_id = c.id
GROUP BY c.name;
```

Tip: Filter and aggregate **before** joining large tables; join on keys of the same datatype.

Set Operations

Operator	Meaning
UNION ALL	Stack results, keep duplicates (fast)
UNION DISTINCT	Stack results and remove duplicates
INTERSECT DISTINCT	Rows common to both
EXCEPT DISTINCT	Rows in the first but not the second

Remember: In BigQuery you must write **ALL** or **DISTINCT** after UNION, INTERSECT and EXCEPT.

Useful Functions

Category	Functions
Aggregate	COUNT, SUM, AVG, MIN, MAX, COUNTIF, ANY_VALUE, ARRAY_AGG, STRING_AGG, APPROX_COUNT_DISTINCT
String	CONCAT, SUBSTR, LOWER, UPPER, TRIM, SPLIT, REPLACE, STARTS_WITH, REGEXP_EXTRACT, FORMAT
Date / time	CURRENT_DATE, DATE_TRUNC, DATE_ADD, DATE_DIFF, EXTRACT, FORMAT_DATE, PARSE_DATE, TIMESTAMP_DIFF
NULL handling	IFNULL, COALESCE, NULLIF, IF, CASE WHEN
Safe versions	SAFE_DIVIDE, SAFE_CAST, SAFE.function() return NULL instead of an error
Conversion	CAST, SAFE_CAST, PARSE_DATE, PARSE_TIMESTAMP
JSON	JSON_VALUE, JSON_QUERY, JSON_EXTRACT_SCALAR, TO_JSON_STRING

EXAMPLES

```
SELECT
  SAFE_DIVIDE(profit, sales) AS margin,
  IFNULL(discount, 0) AS discount,
  DATE_TRUNC(order_date, MONTH) AS m,
  FORMAT_DATE("%Y-%m", order_date) AS ym,
  COUNTIF(status = "Returned") AS rets,
  CASE WHEN amount > 1000
       THEN "High" ELSE "Low" END AS tier
FROM my_ds.sales;
```

Why SAFE_? A normal divide by zero stops the query with an error; **SAFE_DIVIDE** returns NULL and the query continues.

Wildcard Tables

QUERY MANY DATE-SHARDED TABLES

```
SELECT *
FROM `my-project.my_ds.events_*`
WHERE _TABLE_SUFFIX BETWEEN "20260101" AND "20260131";
```

Note: Wildcards help with old **sharded** tables like events_20260101. For new designs prefer one **partitioned** table.

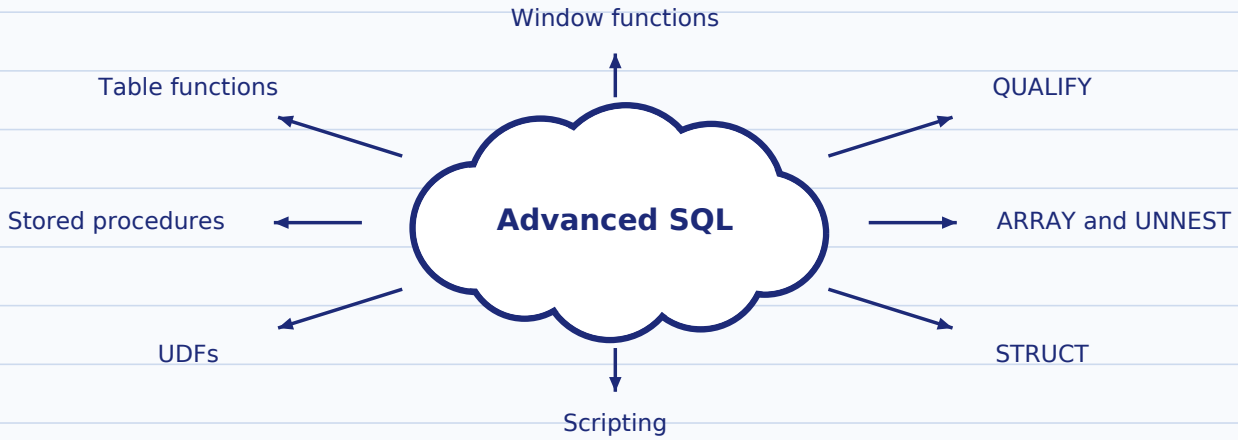
Practice Questions

- Write a query for total sales per region with more than 10,000.
- Difference between WHERE and HAVING.
- What is a CTE?
- What does SELECT * EXCEPT do?
- Why use SAFE_DIVIDE?

7. Advanced SQL

Windows, arrays, scripting, UDFs

Key Ideas



Window (Analytic) Functions

A window function calculates across a set of related rows **without collapsing** them (unlike GROUP BY).

EXAMPLE

```
SELECT region, order_id, amount,
  ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS rn,
  RANK()       OVER (PARTITION BY region ORDER BY amount DESC) AS rk,
  SUM(amount)  OVER (PARTITION BY region)                      AS region_total,
  LAG(amount)  OVER (PARTITION BY region ORDER BY order_date)  AS prev_amount,
  AVG(amount)  OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM my_ds.sales;
```

Function	What it does
ROW_NUMBER()	Unique number 1, 2, 3 ... per partition
RANK()	Rank with gaps after ties (1, 1, 3)
DENSE_RANK()	Rank without gaps (1, 1, 2)
LAG() / LEAD()	Value from the previous / next row
FIRST_VALUE() / LAST_VALUE()	First / last value in the window
SUM() / AVG() OVER	Running totals and moving averages
NTILE(n)	Split rows into n buckets

QUALIFY - Filter on Window Results

WITH QUALIFY

```sql
-- top 3 orders per region
SELECT *
FROM my_ds.sales
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY region
  ORDER BY amount DESC) <= 3;
```

DEDUPLICATE

```sql
-- remove duplicates, keep latest
SELECT *
FROM my_ds.customers
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY id
  ORDER BY updated_at DESC) = 1;
```

Remember: **QUALIFY** is to window functions what HAVING is to GROUP BY - no sub-query needed.

Arrays and Structs

CREATE + AGGREGATE

```sql
SELECT [1, 2, 3] AS nums,
       STRUCT("Asha" AS name, 25 AS age) AS p;

SELECT ARRAY_AGG(product) AS items
FROM my_ds.orders
GROUP BY order_id;      -- rows into array

SELECT ARRAY_LENGTH(items) FROM t;
SELECT p.name FROM t;   -- dot for struct
```

UNNEST + GENERATE

```sql
SELECT order_id, item
FROM my_ds.orders,
     UNNEST(items) AS item;

-- keep rows with empty arrays
SELECT order_id, item
FROM my_ds.orders
LEFT JOIN UNNEST(items) AS item;

SELECT GENERATE_DATE_ARRAY(
  "2026-01-01","2026-01-07");
```

Scripting (Multi-statement Queries)

VARIABLES + IF

```sql
DECLARE start_date DATE DEFAULT DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY);
DECLARE cnt INT64;

SET cnt = (SELECT COUNT(*) FROM my_ds.sales WHERE order_date >= start_date);

IF cnt = 0 THEN
  SELECT "no sales this week";
ELSE
  SELECT cnt AS sales_count;
END IF;
```

- Also available: **LOOP / WHILE / FOR**, **BEGIN ... EXCEPTION ... END** (error handling), **transactions** (BEGIN TRANSACTION / COMMIT).
- Use **CREATE TEMP TABLE** for intermediate results that live only for the script.

UDFs, Procedures and Table Functions

SQL UDF

```
CREATE TEMP FUNCTION add_tax(x FLOAT64)
AS (x * 1.18);

CREATE FUNCTION my_ds.clean(s STRING)
RETURNS STRING
AS (LOWER(TRIM(s)));

SELECT my_ds.clean(name) FROM t;
```

STORED PROCEDURE

```
CREATE PROCEDURE my_ds.daily(IN d DATE)
BEGIN
  DELETE FROM my_ds.agg WHERE day = d;
  INSERT INTO my_ds.agg
  SELECT d, SUM(amount)
  FROM my_ds.sales
  WHERE order_date = d;
END;
CALL my_ds.daily(CURRENT_DATE());
```

Routine	Returns	Use
SQL UDF	One value	Reusable expression
JavaScript UDF	One value	Logic SQL cannot express (slower)
Stored procedure	Nothing (runs statements)	Multi-step jobs, called with CALL
Table function	A table	Parameterized view

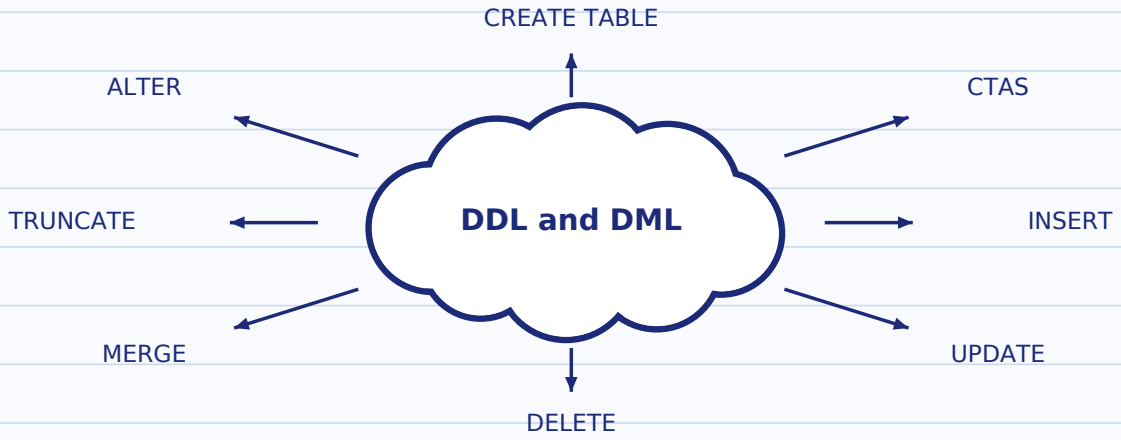
Practice Questions

- Difference between RANK and DENSE_RANK.
- How do you get the top 3 rows per group?
- What does QUALIFY do?
- What is UNNEST used for?
- UDF vs stored procedure.

8. DDL and DML

Create, modify, merge

Key Ideas



DDL - Defining Objects

SCHEMA / TABLE / CTAS

```
CREATE SCHEMA my_ds
OPTIONS(location = "asia-south1");

CREATE TABLE my_ds.customers (
  id INT64 NOT NULL,
  name STRING,
  created DATE
);

CREATE OR REPLACE TABLE my_ds.t AS
SELECT * FROM my_ds.s;      -- CTAS
```

ALTER / DROP

```
CREATE TABLE IF NOT EXISTS my_ds.t (...);

ALTER TABLE my_ds.t
ADD COLUMN email STRING;

ALTER TABLE my_ds.t
DROP COLUMN email;

DROP TABLE my_ds.t;
CREATE VIEW my_ds.v AS SELECT ...;
```

Note: **CTAS** (CREATE TABLE AS SELECT) is the standard way to build a derived table. It is a query job, so it is billed for bytes scanned.

DML - Changing Data

INSERT / UPDATE

```
INSERT INTO my_ds.t (id, name)
VALUES (1, "Asha"), (2, "Ravi");

INSERT INTO my_ds.t
SELECT id, name FROM my_ds.stg;

UPDATE my_ds.t
SET name = "Asha K"
WHERE id = 1;
```

DELETE / TRUNCATE

```
DELETE FROM my_ds.t
WHERE id = 2;

TRUNCATE TABLE my_ds.t;    -- remove all rows

-- delete old rows safely
DELETE FROM my_ds.logs
WHERE log_date < DATE_SUB(CURRENT_DATE(),
                          INTERVAL 90 DAY);
```

MERGE - Upsert

UPSERT: update existing, insert new

```
MERGE my_ds.customers AS T
USING my_ds.stg_customers AS S
ON T.id = S.id
WHEN MATCHED THEN
  UPDATE SET T.email = S.email, T.name = S.name
WHEN NOT MATCHED THEN
  INSERT (id, name, email) VALUES (S.id, S.name, S.email)
WHEN NOT MATCHED BY SOURCE THEN
  DELETE;
```

Remember: MERGE does **INSERT + UPDATE + DELETE** in one atomic statement - ideal for loading changes (CDC) from a staging table.

DML Good Practices

- BigQuery is built for **append and bulk** operations; frequent single-row UPDATE / DELETE is slow and costly.
- Prefer **batch** DML (MERGE, bulk INSERT) over many small statements.
- Use **partitioned** tables so DML touches only some partitions.
- DML is ACID and snapshot-isolated; you can wrap statements in **multi-statement transactions**.
- Recently streamed rows may be restricted from DML for a short time (streaming buffer).

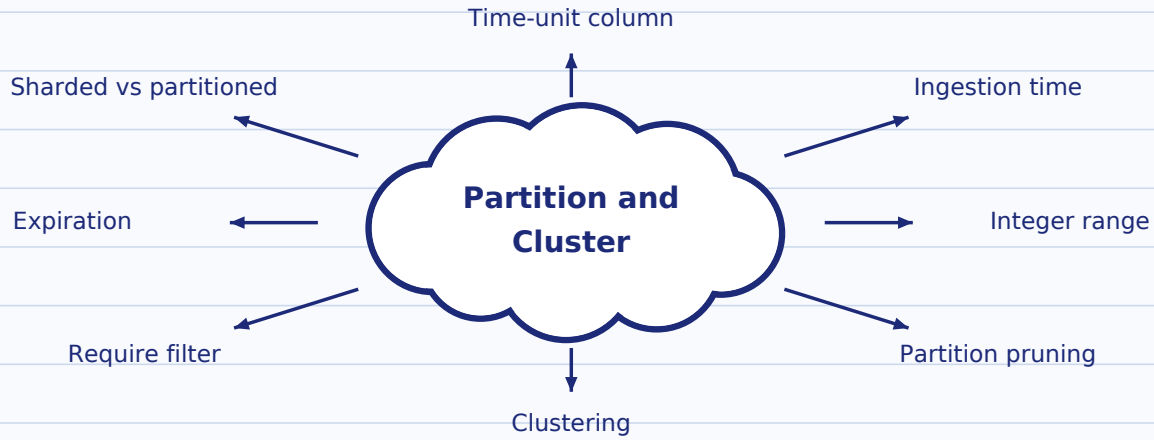
Practice Questions

- What is CTAS?
- Explain MERGE with an example.
- Why avoid many single-row UPDATE statements?
- Difference between DELETE and TRUNCATE.
- What is DDL vs DML?

9. Partitioning and Clustering

Scan less, pay less

Key Ideas



Partitioning

Splits a table into **segments (partitions)**. A query that filters on the partition column reads only the matching partitions - this is called **partition pruning**.

Type	Partitioned by	Example
Time-unit column	A DATE / TIMESTAMP / DATETIME column (hour, day, month, year)	`PARTITION BY order_date`
Ingestion time	Time the row was loaded (pseudo column _PARTITIONTIME)	`PARTITION BY _PARTITIONDATE`
Integer range	Ranges of an INT64 column	`PARTITION BY RANGE_BUCKET(id, GENERATE_ARRAY(0,1000,100))`

PARTITIONED + CLUSTERED TABLE

```
CREATE TABLE my_ds.sales (
  order_id INT64, order_date DATE, region STRING, amount NUMERIC)
PARTITION BY order_date
CLUSTER BY region
OPTIONS (
  partition_expiration_days = 365,
  require_partition_filter  = TRUE);
```

- A table can have up to **10,000 partitions**; only **one** partition column is allowed.
- **require_partition_filter** forces every query to filter on the partition column - prevents accidental full scans.
- **partition_expiration_days** deletes old partitions automatically.
- Each partition becomes **long-term storage** separately after 90 days unmodified.

Clustering

Sorts data **within** a table (or partition) by up to **4 columns**, so BigQuery can skip blocks of data that do not match filters.

- **Column order matters**: the first cluster column is sorted first.
- Best on columns used often in **WHERE, GROUP BY, JOIN** with high cardinality (customer_id, region, product).
- BigQuery **re-clusters automatically** in the background at no charge.
- Works on partitioned and non-partitioned tables.
- Cost saving is known only **after** the query runs (estimate is an upper bound).

	Partitioning	Clustering
Columns	One	Up to 4
Cardinality	Low (days, months)	High is fine
Cost estimate before query	**Exact** (dry run)	Upper bound only
Limit	10,000 partitions	4 columns
Use together?	Yes - partition by date, cluster by id / region	

Golden Rule: Partition on **date**, cluster on the **columns you filter by most**. Together they cut scanned bytes (and cost) dramatically.

Partitioned vs Sharded Tables

	Sharded (events_20260101)	Partitioned (one table)
Metadata	Each table has its own - slower, many tables	Single table - **faster**
Management	Many objects to secure and maintain	One table, one set of permissions
Querying	Needs wildcard + _TABLE_SUFFIX	Normal WHERE on the partition column
Recommended	No	**Yes**

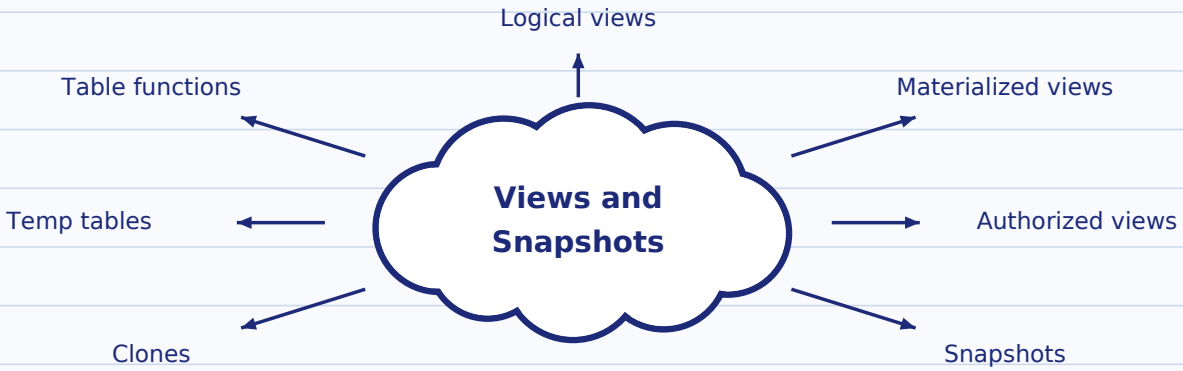
Practice Questions

- What is partition pruning?
- Types of partitioning in BigQuery.
- How many columns can you cluster by?
- Partitioning vs clustering - when to use which?
- Why prefer partitioned tables over sharded tables?

10. Views, Snapshots and Clones

Reusable and point-in-time objects

Key Ideas



Views

Type	What it is	Key points
Logical view	Saved SQL query, runs every time you query it	No stored data; billed on the underlying tables scanned
Materialized view	Pre-computed result stored and **auto-refreshed**	Faster and cheaper; smart rewriting of matching queries; limited SQL (aggregations, simple joins)
Authorized view	View that can read a dataset the user cannot access	Share filtered / masked data without sharing the base table

LOGICAL VIEW

```
CREATE VIEW my_ds.v_sales AS
SELECT region, SUM(amount) AS total
FROM my_ds.sales
GROUP BY region;
```

MATERIALIZED VIEW

```
CREATE MATERIALIZED VIEW my_ds.mv_sales
AS
SELECT region, SUM(amount) AS total
FROM my_ds.sales
GROUP BY region;
```

Remember: Materialized views refresh automatically (default about every 30 minutes) and BigQuery only recomputes the **changed** data - so you pay much less than re-running a heavy query.

Snapshots and Clones

	Table snapshot	Table clone
Mode	**Read-only**	**Read / write**
Purpose	Backup / point-in-time copy	Safe copy for dev, testing, experiments
Storage cost	Only data that differs from the base table	Only data that differs from the base table
Syntax	`CREATE SNAPSHOT TABLE s CLONE t`	`CREATE TABLE c CLONE t`

Key Point: Both are **copy-on-write**: creating them is instant and costs almost nothing until data changes.

Temporary Tables and Table Functions

- **Anonymous table**: every query result is stored for about 24 hours (this powers the query cache).
- **CREATE TEMP TABLE**: lives only for the script or session.
- **Table function**: a parameterized view that returns a table: `CREATE TABLE FUNCTION ds.f(d DATE) AS (SELECT ...)`.

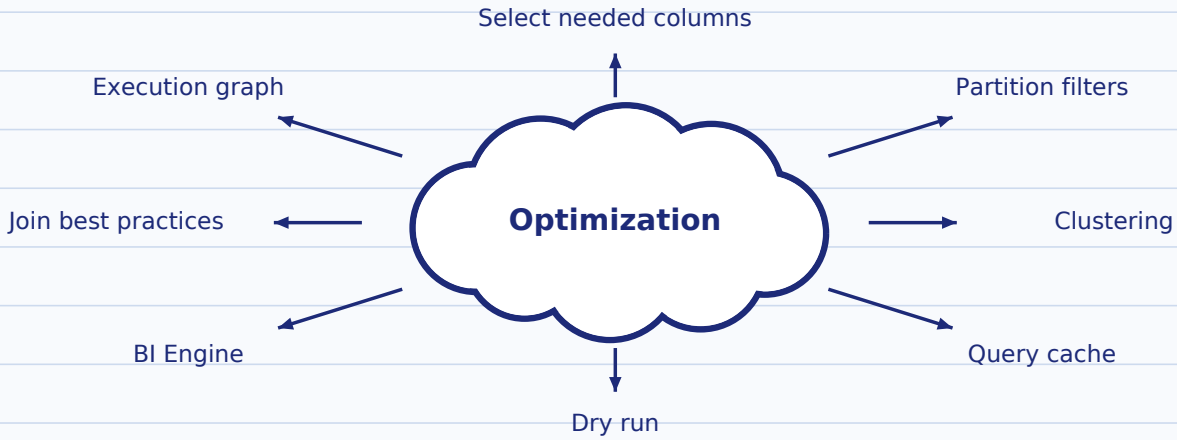
Practice Questions

- Logical view vs materialized view.
- What is an authorized view?
- Snapshot vs clone.
- Why are snapshots cheap?

11. Performance and Cost Optimization

Make queries faster and cheaper

Key Ideas



Golden Rules

Do	Why
Select only the **columns you need**	Columnar storage bills by columns scanned; avoid SELECT *
Filter on partition column	Partition pruning skips whole partitions
Use **clustering** on filter / join columns	Skips blocks of data inside partitions
Filter and aggregate **before joins**	Smaller data is shuffled
Prefer **nested / repeated** (denormalized) data	Fewer expensive joins
Use **approximate** functions for huge data	APPROX_COUNT_DISTINCT is far cheaper than COUNT(DISTINCT)
Avoid **CROSS JOIN** and self-joins on big tables	They explode row counts
Avoid JavaScript UDFs when SQL works	SQL is much faster
Order by / LIMIT only at the **end**	Sorting early wastes slots
Use **materialized views** for repeated aggregates	Pre-computed and incremental

Myth: **LIMIT does not reduce cost** on a normal table - BigQuery still scans all selected columns. Use partition filters and column selection instead.

Estimating and Controlling Cost

DRY RUN + MAX BYTES

```
-- CLI dry run: shows bytes, runs nothing
bq query --dry_run --use_legacy_sql=false \
  "SELECT region FROM my_ds.sales"

-- hard cap on a query
bq query --maximum_bytes_billed=1000000000 ...
```

Controls

- **Dry run**: free, shows bytes to be processed (also in the console editor)
- **Maximum bytes billed**: query fails if it would exceed the limit
- **Custom quotas**: daily cap per project / user
- **Budgets and alerts** in Cloud Billing
- **require_partition_filter** on big tables
- **Labels** to track cost per team

Query Cache and BI Engine

	Query cache	BI Engine
What	Stores results of an identical query	In-memory acceleration layer for BI queries
Cost	**Free** when a cached result is used	Paid reservation (GB of memory)
Duration	About 24 hours	As long as capacity is reserved
Conditions	Same query text, tables unchanged, no non-deterministic functions (e.g. RAND, CURRENT_TIMESTAMP)	Works with Looker Studio, Looker, Sheets, many BI tools

Reading the Query Plan

- The console **Execution graph** shows each stage: slots used, time, rows read / written, **shuffle** size.
- Look for stages with huge **skew** (one slow worker), big shuffles, or heavy sorts.
- **Bytes processed vs bytes billed**: billed rounds up (minimum 10 MB per table).
- Query **INFORMATION_SCHEMA.JOBS** to find your most expensive queries over time.

TOP 10 COSTLY QUERIES (LAST 7 DAYS)

```
SELECT user_email, query,
       total_bytes_billed / POW(1024, 4) AS tib_billed
FROM `region-us`.INFORMATION_SCHEMA.JOBS
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
ORDER BY total_bytes_billed DESC
LIMIT 10;
```

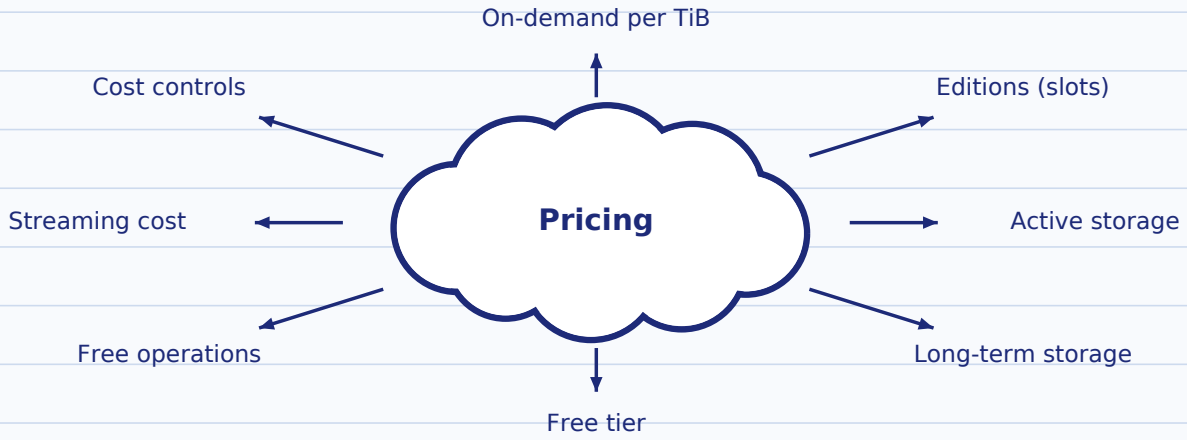
Practice Questions

- Does LIMIT reduce BigQuery cost? Why?
- List 6 ways to optimize query cost.
- What is a dry run?
- When is the query cache used?
- What is BI Engine?

12. BigQuery Pricing

Compute + storage

Key Ideas



Two Parts of the Bill

Compute
queries (analysis)

→

Storage
active + long-term

→

Extras
streaming, BI Engine, ML

Note: Prices below are approximate **US list prices** and change; always confirm on the Google Cloud pricing page and use the pricing calculator.

Compute Pricing Models

	On-demand (default)	Capacity / Editions
You pay	About **$6.25 per TiB** of data scanned (US)	**Slot-hours** of reserved / autoscaled capacity
Free tier	**First 1 TiB per month free**	None
Predictability	Varies with the data your queries scan	**Predictable** budget
Best for	Light or spiky use, small teams	Heavy, steady workloads, many users
Commitments	None	Optional 1-year / 3-year (Enterprise tiers)

Edition	For
Standard	Basic workloads; autoscaling, no commitments
Enterprise	Production analytics: more security / governance, commitments, BigQuery ML features
Enterprise Plus	Mission-critical: highest availability, compliance, advanced controls

Remember: Editions bill per slot-hour with **autoscaling**. The older flat-rate model has been replaced; the pricing model is picked per reservation assignment.

Storage Pricing

Storage	Meaning	Approx. price
Active	Table / partition modified in the last **90 days**	About $0.02 per GB-month (logical)
Long-term	Not modified for **90 consecutive days**	About $0.01 per GB-month (roughly half)
First 10 GiB	Free every month	$0

- Long-term storage is automatic; **same performance, durability and availability** as active.
- Each **partition** of a partitioned table ages separately.
- You can choose **logical** (uncompressed) or **physical** (compressed) storage billing per dataset - physical is cheaper when data compresses well, but counts time-travel and fail-safe bytes.
- Storage is billed per second, prorated.

Free and Paid Operations

Free	Paid
Batch **load**, **export**, **copy** jobs (shared slots)	Queries (on-demand per TiB or slots)
Metadata operations, dry runs, cached results	Streaming inserts / Storage Write API (first ~2 TiB a month free)
Table / dataset creation, listing, deleting	BI Engine, BigQuery ML training and some model types, Omni
First 1 TiB queried and 10 GiB stored every month	Storage above the free 10 GiB

QUICK COST MATH

```
-- Example: 5 TiB scanned in the month, on-demand
(5 - 1 free) x $6.25 = $25.00

-- Example: 500 GB active storage
500 x $0.02 = about $10 per month
```

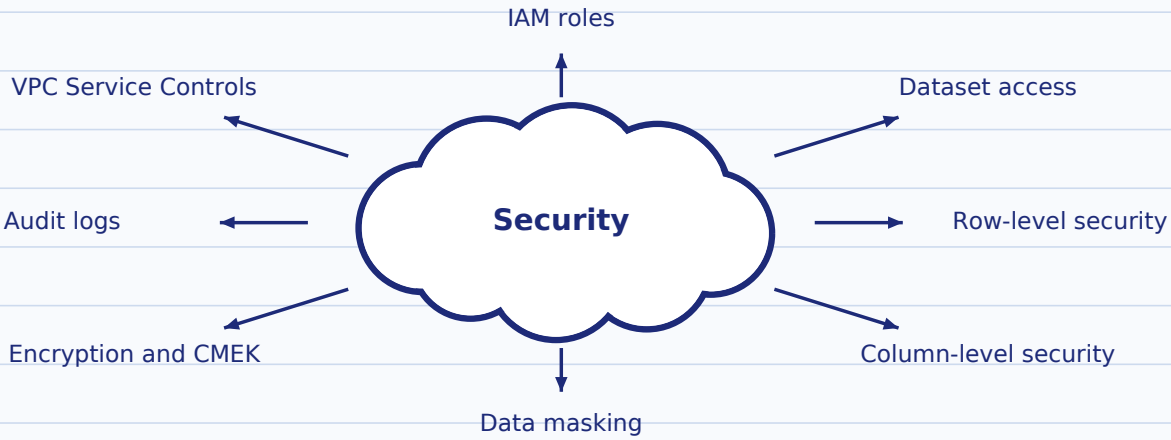
Practice Questions

- Explain on-demand vs capacity pricing.
- What are the three Editions?
- Active vs long-term storage - when does the change happen?
- Which operations are free?
- How do you estimate a query cost?

13. Security and Governance

Access, privacy, encryption, audit

Key Ideas



IAM Roles

Role	Can do
BigQuery Admin	Everything: manage all resources and data
Data Owner	Full control of datasets and tables, can grant access
Data Editor	Read and write table data, create tables
Data Viewer	Read data and metadata
Job User	Run jobs (queries) in the project
User	Run jobs and create datasets
Metadata Viewer	See metadata only, not data
Read Session User	Use the Storage Read API

Golden Rule: To query a table a user needs **Data Viewer** on the data AND **Job User** (or User) on a project to run and bill the job. Grant roles at the lowest level possible and to **groups**.

Fine-grained Access

Level	Mechanism
Project / dataset / table	IAM roles at that resource
Column-level	**Policy tags** (taxonomies in Dataplex / Data Catalog); only users with Fine-Grained Reader see the column
Row-level	Row access policies filter rows per user / group
Data masking	Dynamic masks show hashed / nulled / partial values to some users
Authorized views / datasets	Share a safe slice without exposing base tables

ROW-LEVEL SECURITY

```
CREATE ROW ACCESS POLICY south_only
ON my_ds.sales
GRANT TO ("group:south-team@company.com")
FILTER USING (region = "South");
```

Encryption, Audit and Perimeters

Area	Feature
Encryption at rest	**Always on** with Google-managed keys; optional **CMEK** (customer-managed keys in Cloud KMS)
Encryption in transit	TLS
Audit	**Cloud Audit Logs**: admin activity, data access, who ran what query
Network	**VPC Service Controls** create a security perimeter around BigQuery
Sensitive data	**Sensitive Data Protection (DLP)** finds and masks PII
Governance	**Dataplex / Data Catalog**: discovery, lineage, quality, tags

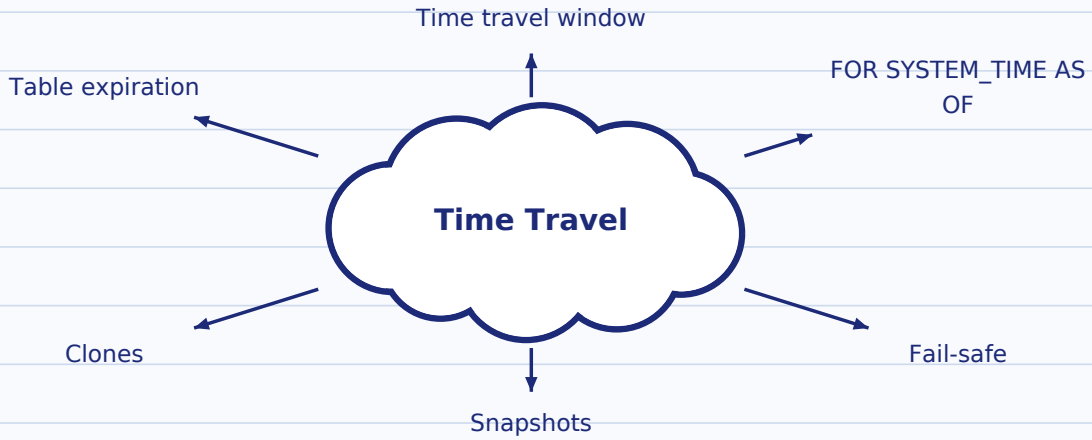
Practice Questions

- Which roles are needed to run a query on a table?
- Row-level vs column-level security.
- What is an authorized view?
- What is CMEK?
- How do you audit who queried a table?

14. Time Travel and Data Recovery

Undo mistakes

Key Ideas



Time Travel

- Lets you query or restore data as it was at any point in the **time travel window**.
- Default window: **7 days**; configurable from **2 to 7 days** per dataset.
- Works for changed or **deleted** tables too (within the window).

QUERY THE PAST

```
SELECT *
FROM my_ds.orders
FOR SYSTEM_TIME AS OF
  TIMESTAMP_SUB(CURRENT_TIMESTAMP(),
                INTERVAL 1 HOUR);
```

RESTORE

```
-- restore a bad delete
CREATE OR REPLACE TABLE my_ds.orders AS
SELECT * FROM my_ds.orders
FOR SYSTEM_TIME AS OF
  TIMESTAMP "2026-10-09 10:00:00+00";
```

Fail-safe

	Time travel	Fail-safe
Duration	2 to 7 days (default 7)	**7 days** after time travel ends
Who can use	You, with SQL	**Google support only**; for emergency recovery
Queryable?	Yes	**No**
Billed?	Yes (with physical storage billing)	Yes (with physical storage billing)

Data changed or deleted → **Time travel window (you can restore)** → **Fail-safe 7 days (Google support)** → **Gone**

- For longer retention use **table snapshots** or **scheduled backups** and set **table expiration** to clean up.

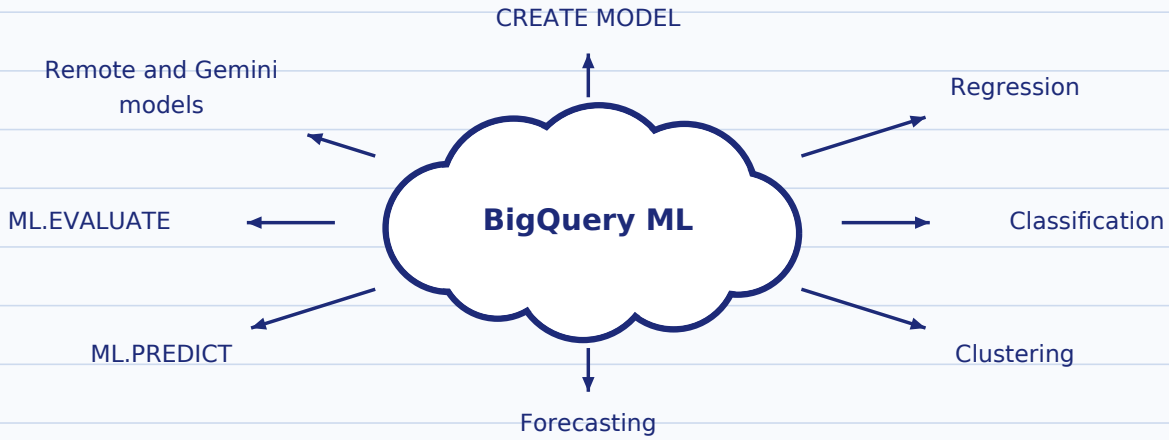
Practice Questions

- What is time travel and how long does it last?
- Time travel vs fail-safe.
- How do you recover an accidentally deleted table?
- Snapshot vs time travel.

15. BigQuery ML

Machine learning with SQL

Key Ideas



What is BigQuery ML?

Create, train, evaluate and use ML models **using only SQL**, right where the data lives - no data export, no separate ML infrastructure.

1. CREATE MODEL (train) → **2. ML.EVALUATE (check quality)** → **3. ML.PREDICT (use the model)**

Model type	Used for
LINEAR_REG	Predict a number (price, sales)
LOGISTIC_REG	Classify yes / no or categories (churn, spam)
KMEANS	Clustering - group similar customers
BOOSTED_TREE / RANDOM_FOREST	Strong regression / classification
DNN models	Deep learning
ARIMA_PLUS	Time-series forecasting
MATRIX_FACTORIZATION	Recommendations
Imported / remote models	TensorFlow, ONNX, Vertex AI and Gemini models called from SQL

Example: Predict Churn

1. TRAIN

```
CREATE OR REPLACE MODEL my_ds.churn_model
OPTIONS (
  model_type = "logistic_reg",
  input_label_cols = ["churned"]) AS
SELECT tenure, monthly_charges,
       contract_type, churned
FROM my_ds.customers;
```

2. EVALUATE + 3. PREDICT

```
SELECT * FROM ML.EVALUATE(
  MODEL my_ds.churn_model);

SELECT customer_id,
       predicted_churned
FROM ML.PREDICT(
  MODEL my_ds.churn_model,
  (SELECT * FROM my_ds.new_customers));
```

Note: Some model types are billed per TiB of data processed in training (higher than normal queries); others use Vertex AI pricing. Check pricing before big runs.

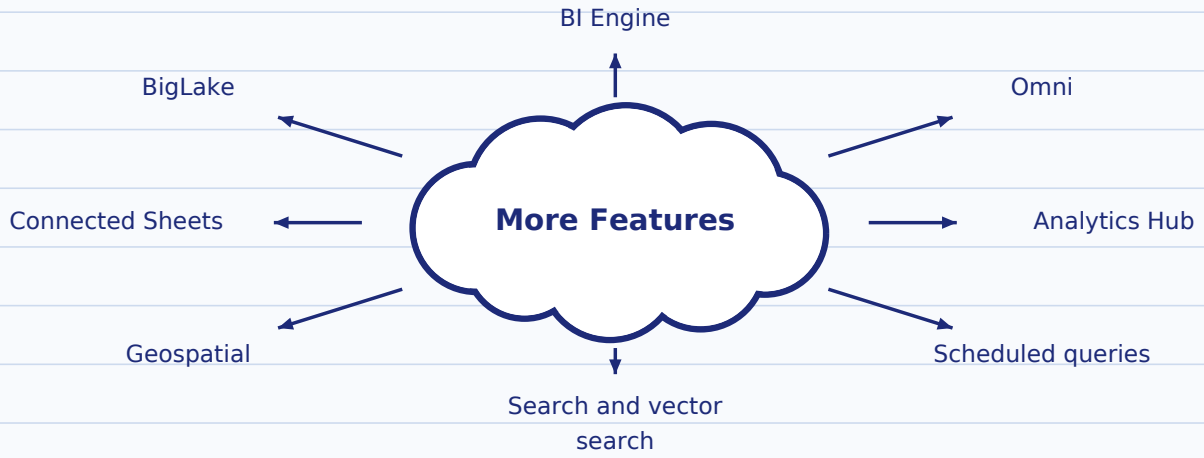
Practice Questions

- What is BigQuery ML?
- Name 5 model types.
- What are ML.EVALUATE and ML.PREDICT?
- Why is BigQuery ML useful for analysts?

16. More BigQuery Features

Sharing, search, geo, multi-cloud

Key Ideas



Feature Guide

Feature	What it does
BI Engine	In-memory acceleration for dashboards (Looker Studio, Looker, Sheets)
BigQuery Omni	Query data in **AWS S3 / Azure Blob** without moving it
Analytics Hub	Share and subscribe to datasets through exchanges and listings (linked datasets)
Scheduled queries	Run SQL on a schedule (built on Data Transfer Service)
Dataform	SQL workflows with dependencies, tests, version control
Search index	Fast text search with the `SEARCH()` function
Vector search	Similarity search on embeddings (`VECTOR_SEARCH`) for AI apps
Geospatial (GIS)	GEOGRAPHY type and ST_ functions: distance, within, intersects
BigLake	Unified, secure access to lake files and open table formats like Iceberg
Connected Sheets	Analyze billions of rows from Google Sheets
BigQuery Studio	Workspace with SQL, Python notebooks, and Gemini assistance
Remote functions	Call Cloud Run / Functions from SQL

GEOSPATIAL: STORES WITHIN 5 KM OF PUNE

```
SELECT name,
  ST_DISTANCE(location, ST_GEOGPOINT(73.85, 18.52)) / 1000 AS km
FROM my_ds.stores
WHERE ST_DWITHIN(location, ST_GEOGPOINT(73.85, 18.52), 5000);
```

Scheduling

• **Scheduled queries**: run a query hourly / daily and write the result to a table (great for refreshing reporting tables).
• **Dataform** or **Cloud Composer (Airflow)** for multi-step pipelines.
• Combine with **materialized views** so dashboards read pre-aggregated data.

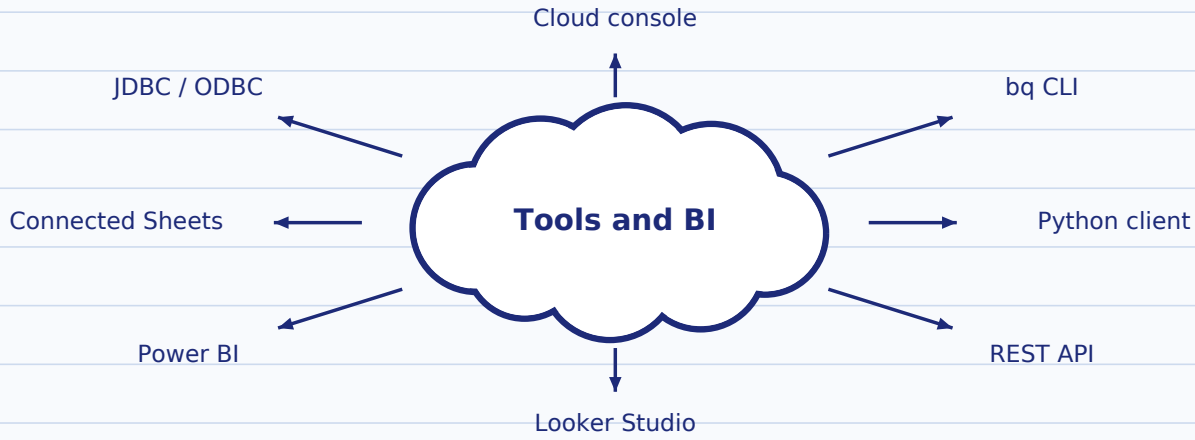
Practice Questions

• What is Analytics Hub?
• What does BigQuery Omni do?
• What are scheduled queries?
• Name two ways BigQuery supports AI search.

17. Tools, APIs and BI Connections

Console, bq, Python, Power BI

Key Ideas



Ways to Work with BigQuery

Tool	Use
Cloud console / BigQuery Studio	Write SQL, browse datasets, view the query plan, run notebooks
bq CLI	Scripts and automation from the command line (Cloud Shell or local)
Client libraries	Python, Java, Go, Node.js, C# etc. for applications and pipelines
REST / RPC API	Direct programmatic access, Storage Read / Write APIs for high throughput
JDBC / ODBC drivers	Connect BI and database tools
Looker Studio, Looker, Sheets	Dashboards and spreadsheets directly on BigQuery

bq Command-line Cheat Sheet

Task	Command
List datasets / tables	`bq ls` , `bq ls my_ds`
Create dataset	`bq mk --dataset --location=asia-south1 my_ds`
Show table / schema	`bq show my_ds.orders` , `bq show --schema my_ds.orders`
Preview rows (free)	`bq head -n 10 my_ds.orders`
Run SQL	`bq query --use_legacy_sql=false "SELECT 1"`
Load file	`bq load --autodetect --source_format=CSV my_ds.t gs://b/f.csv`
Copy table	`bq cp my_ds.a my_ds.b`
Export table	`bq extract my_ds.t gs://b/out-*.csv`
Delete table	`bq rm -t my_ds.t`

Python Client

QUERY INTO A PANDAS DATAFRAME

```
from google.cloud import bigquery

client = bigquery.Client(project="my-project")
sql = "SELECT region, SUM(amount) AS total FROM `my_ds.sales` GROUP BY region"

df = client.query(sql).to_dataframe()
print(df.head())
```

LOAD A DATAFRAME

```
# write a DataFrame back to BigQuery
job = client.load_table_from_dataframe(
    df, "my-project.my_ds.region_totals")
job.result()          # wait for the load
```

Setup: `pip install google-cloud-bigquery pandas db-dtypes` and authenticate with **gcloud auth application-default login**.

Connecting Power BI to BigQuery

1. Power BI Desktop Get Data → **2. Google BigQuery connector** → **3. Sign in with Google account** → **4. Pick project, dataset, table** → **5. Import or DirectQuery**

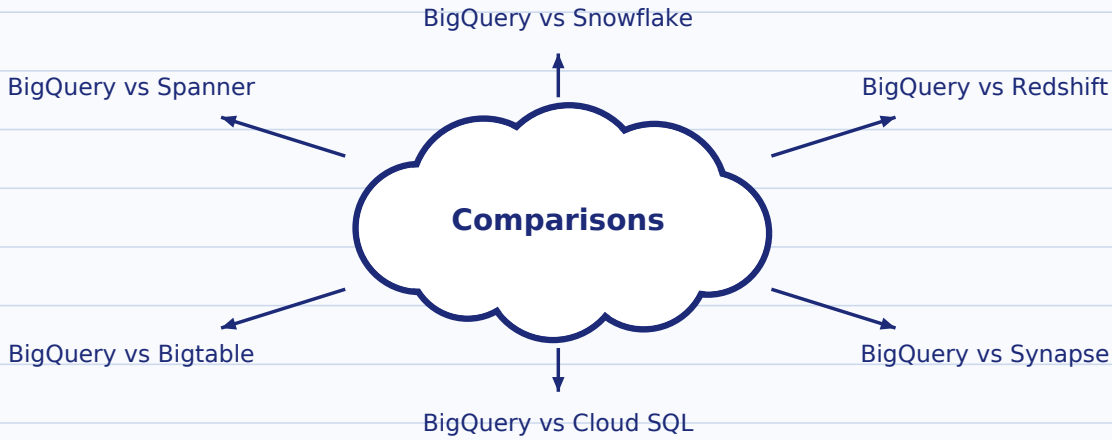
	Import	DirectQuery
Data	Copied into Power BI (refresh on schedule)	Stays in BigQuery; every visual sends a query
Speed	Fast (in-memory)	Depends on BigQuery
BigQuery cost	Only at refresh	**Every interaction** can scan data
Best for	Aggregated / small-to-medium data	Very large or near real-time data

- The user needs **BigQuery Data Viewer** on the data and **Job User** on a project to bill the queries (set it as the **Billing Project** in connector options).
- Point Power BI at **views or aggregated tables**, not raw huge tables; avoid SELECT *.
- Use **partition filters** in custom SQL and **incremental refresh** with Import mode to keep scans small.
- Consider **BI Engine** and **materialized views** to speed up and cheapen dashboards.

18. BigQuery vs Other Services

Pick the right tool

Key Ideas



BigQuery vs Other Data Warehouses

	BigQuery	Snowflake	Amazon Redshift	Azure Synapse / Fabric
Cloud	Google Cloud	AWS, Azure, GCP	AWS	Azure
Model	Fully serverless	Virtual warehouses (you size them)	Provisioned clusters or Serverless	Dedicated pools or serverless / Fabric capacity
Billing	Per TiB scanned or slot-hour; storage separate	Credits for compute + storage	Node-hours or RPU-hours + storage	Capacity units / DWUs + storage
Strength	Zero ops, built-in ML, streaming, GCP integration	Multi-cloud, data sharing	AWS ecosystem	Microsoft ecosystem, Power BI

BigQuery vs Other GCP Databases

Service	Type	Use for
BigQuery	Analytical data warehouse (OLAP)	SQL analytics, BI, ML on huge data
Cloud SQL	Managed MySQL / PostgreSQL / SQL Server (OLTP)	Standard application databases
AlloyDB	PostgreSQL-compatible, high performance	Demanding PostgreSQL apps, hybrid transactional / analytical
Spanner	Globally distributed relational (OLTP)	Huge-scale transactional apps with strong consistency
Bigtable	Wide-column NoSQL	Very high-throughput, low-latency (IoT, time-series)
Firestore	Document NoSQL	Mobile / web app data
Cloud Storage	Object storage	Data lake, raw files, backups

Remember: **Transactions go to Cloud SQL / Spanner. Analytics go to BigQuery.** A common pattern: operational databases feed BigQuery for reporting.

Practice Questions

- BigQuery vs Cloud SQL.
- BigQuery vs Bigtable.
- Compare BigQuery with Snowflake and Redshift.

19. Interview Questions and Answers

Quick revision

Top BigQuery Questions (1 - 13)

#	Question	Answer
1	What is BigQuery?	Serverless, fully managed, columnar, petabyte-scale data warehouse on GCP queried with SQL.
2	Why is it fast?	Columnar storage, massive parallelism (Dremel), separate storage and compute, fast Jupiter network.
3	What is a slot?	Unit of compute (CPU, memory, I/O) used to run query stages.
4	On-demand vs capacity pricing?	On-demand: per TiB scanned (1 TiB/month free). Capacity: per slot-hour (Editions) for predictable cost.
5	Active vs long-term storage?	Active: modified in last 90 days. Long-term: untouched 90 days, about half price, same performance.
6	How to reduce query cost?	Select needed columns, partition filter, clustering, materialized views, dry run, max bytes billed, cache.
7	Partitioning vs clustering?	Partition splits table by one column (exact pruning). Cluster sorts by up to 4 columns within it.
8	Does LIMIT cut cost?	No. Bytes scanned (columns and partitions) decide the cost.
9	What is a materialized view?	Pre-computed, auto-refreshed query result; faster and cheaper than the base query.
10	Native vs external table?	Native data lives in BigQuery storage. External reads files in place (slower, fewer features).
11	What is nested and repeated data?	STRUCT and ARRAY columns; denormalizes one-to-many data and avoids joins.
12	What is time travel?	Query or restore data as of any point in the last 2 to 7 days (default 7).
13	Snapshot vs clone?	Snapshot is read-only backup; clone is writable copy; both store only changed data.

Top BigQuery Questions (14 - 26)

#	Question	Answer
14	How do you load data?	Batch load (free), streaming / Storage Write API, Data Transfer Service, Dataflow, external tables.
15	Best file formats?	Avro and Parquet (self-describing, compressed). CSV and newline-delimited JSON also supported.
16	What is MERGE?	Single statement to insert, update and delete based on a join with a source table (upsert).
17	What is QUALIFY?	Filters results of window functions, like HAVING for GROUP BY.
18	Row-level vs column-level security?	Row access policies filter rows; policy tags restrict columns. Authorized views share safe slices.
19	Roles to run a query?	Data Viewer on data + Job User (or User) on the billing project.
20	What is BigQuery ML?	Create and run ML models with SQL (CREATE MODEL, ML.PREDICT).
21	What is BI Engine?	In-memory acceleration layer for BI dashboards.
22	What is Analytics Hub?	Data-sharing service using exchanges and listings.
23	Is BigQuery OLTP?	No. It is OLAP; use Cloud SQL or Spanner for transactions.
24	What are free operations?	Load, export, copy, metadata, cached queries, dry runs, first 1 TiB queried and 10 GiB stored.
25	How to find costly queries?	INFORMATION_SCHEMA.JOBS (total_bytes_billed) and billing reports with labels.
26	Dataset location rule?	Fixed at creation; queries can only join datasets in the same location.

20. Quick Revision Cheat Sheet

Numbers and one-liners to remember

Numbers to Remember

Fact	Value
Free queries every month	First **1 TiB** processed
Free storage every month	First **10 GiB**
On-demand query price (US, approx.)	**$6.25** per TiB scanned
Active storage (approx.)	About **$0.02** per GB-month
Long-term storage	After **90** days unmodified, about half price
Time travel	**2 to 7** days (default 7); fail-safe **7** more days
Max partitions per table	**10,000**
Max clustering columns	**4**
Query cache / anonymous tables	About **24 hours**
On-demand slots per project (default)	About **2,000**

One-line Summary of Everything

Topic	Remember
What	Serverless columnar data warehouse for SQL analytics
Architecture	Dremel + Colossus + Capacitor + Jupiter + Borg, storage and compute separated
Hierarchy	Project > Dataset > Table / View / Routine / Model
Types	INT64, FLOAT64, NUMERIC, STRING, DATE, TIMESTAMP, ARRAY, STRUCT, JSON, GEOGRAPHY
Loading	Batch (free), streaming / Write API, Data Transfer Service, external tables
SQL	GoogleSQL, CTEs, window functions, QUALIFY, UNNEST, MERGE, scripting, UDFs
Speed and cost	Partition + cluster, select columns, materialized views, dry run, caps
Pricing	On-demand per TiB or Editions per slot-hour; storage active vs long-term
Security	IAM, row / column security, masking, CMEK, audit logs, VPC-SC
Recovery	Time travel, fail-safe, snapshots, clones
Extras	BigQuery ML, BI Engine, Omni, Analytics Hub, search, GIS

Golden Rule: Scan less data = pay less and run faster. Everything in BigQuery optimization comes back to **reading fewer columns and fewer partitions**.